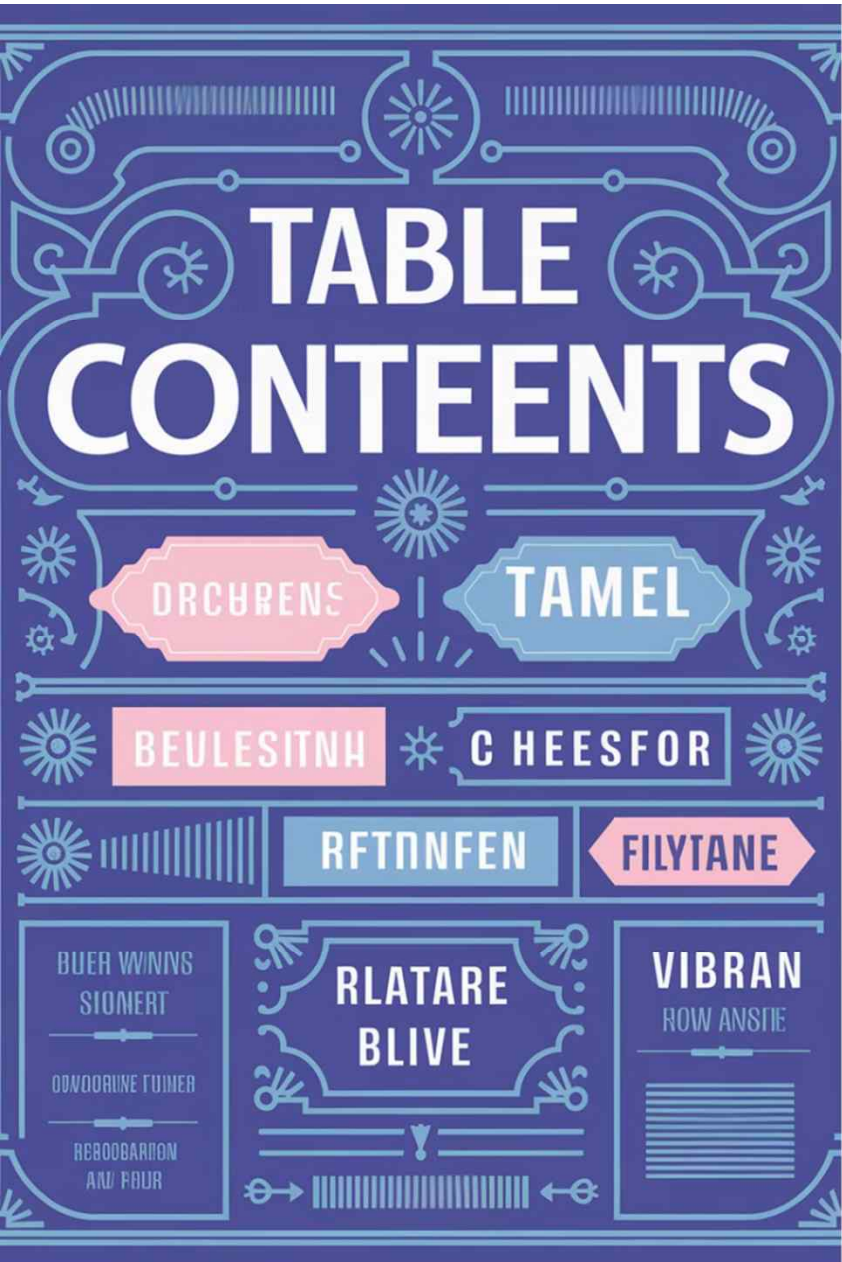




Django 소개: 웹 개발의 혁명

장고는 웹 개발의 판도를 바꾼 혁신적인 프레임워크입니다. 파이썬 기반으로 동작하며 빠르고 안전한 웹사이트를 구축할 수 있습니다.



목차

- 1 소개
Django의 기본 개념과 역사
- 2 기본 기능
프로젝트 설정, 모델, 뷰, 템플릿
- 3 고급 기능
REST API, 캐싱, 테스트, 배포
- 4 확장과 최적화
성능 개선, 서드파티 통합, 마이크로서비스

Django란 무엇인가?

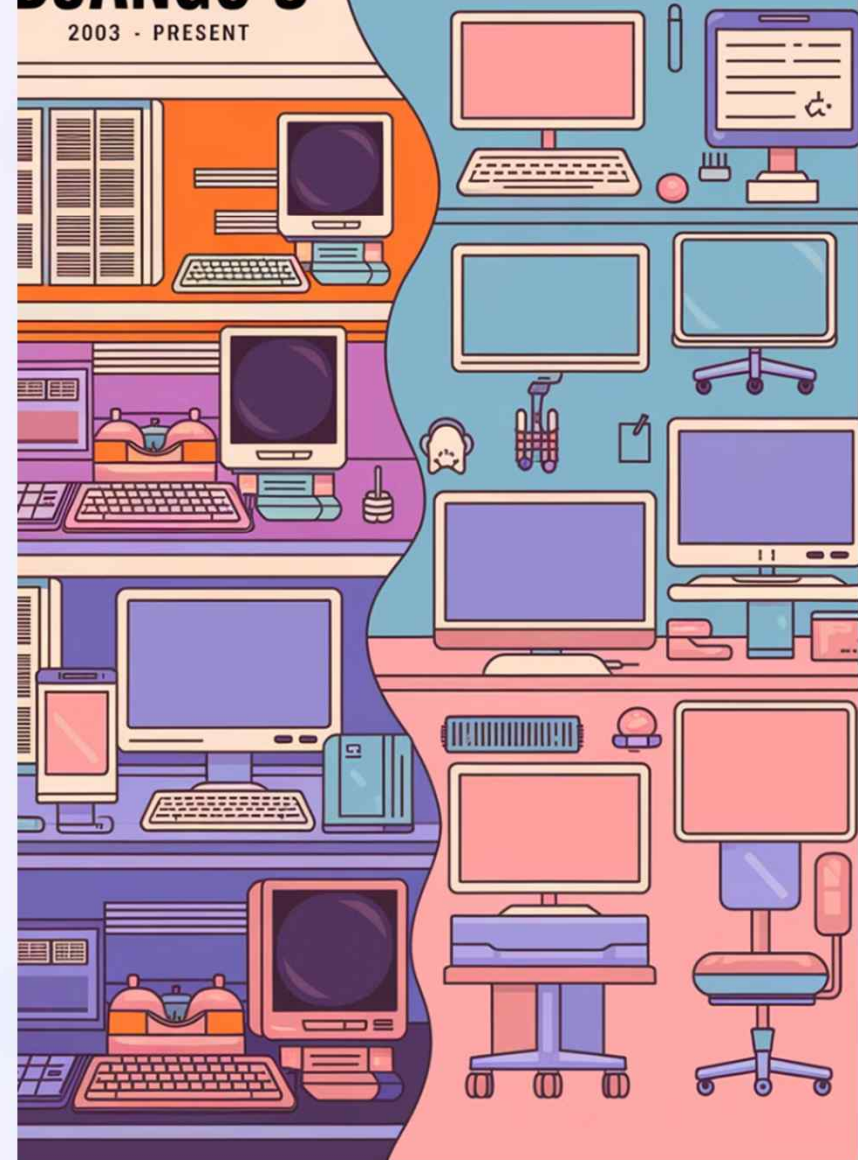
- 1 파이썬 기반 웹 프레임 워크
파이썬의 간결함과 강력함을 웹 개발에 적용한 프레임워크입니다.
- 2 MVC 아키텍처 기반
모델-뷰-컨트롤러 패턴을 구현해 코드의 구조화를 돕습니다.
- 3 빠른 개발과 깔끔한 디자인
반복 작업을 최소화하여 개발 속도를 높이고 유지보수성을 개선합니다.

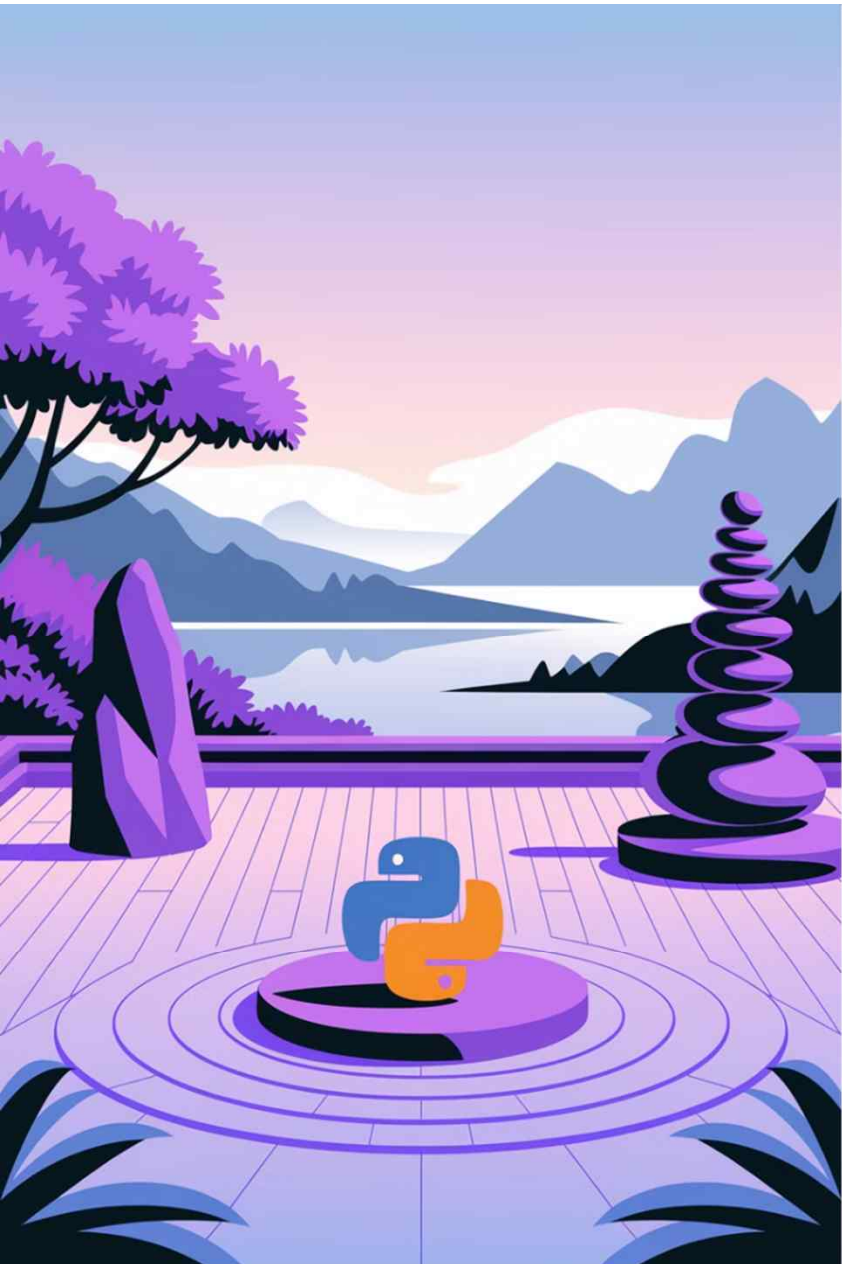


Django의 역사

- 1 2003년
Lawrence Journal-World 신문사에서 웹 애플리케이션 개발 시작
- 2 2005년
BSD 라이선스 하에 오픈소스로 공개
- 3 2008년
Django 소프트웨어 재단 설립
- 4 현재
버전 4.2 출시, 전 세계적으로 사용되는 인기 프레임워크

DJANGO'S
2003 - PRESENT





Django의 철학



DRY 원칙

Don't Repeat Yourself,
코드 반복을 최소화하
여 효율성 극대화



명시적 선호

암시적인 것보다 명시
적인 것이 더 좋다는
원칙 추구



빠른 개발

생산성을 극대화하여
신속한 웹 애플리케이
션 개발 지원

Django 설치하기

1

가상환경 생성

python -m venv myenv 명령으로 격리된 환경을 만듭니다.

2

가상환경 활성화

Windows: myenv\Scripts\activate, Mac/Linux: source myenv/bin/activate

3

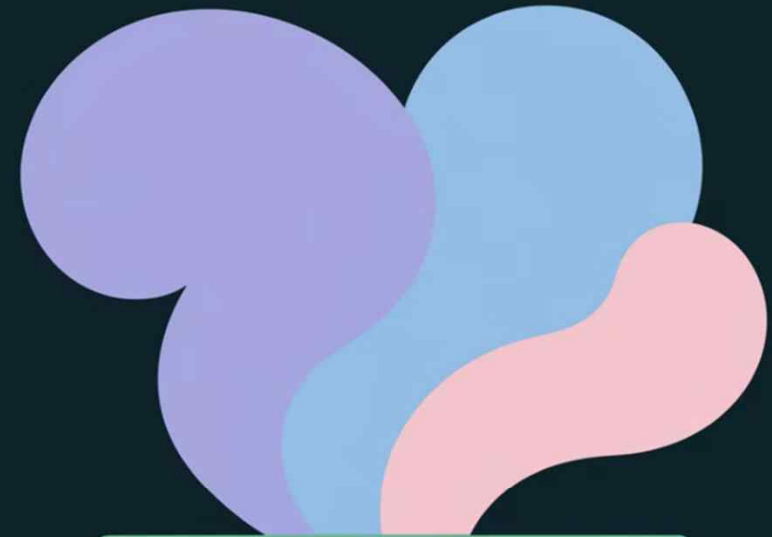
Django 설치

pip install django 명령으로 최신 버전을 설치합니다.

4

버전 확인

python -m django --version 명령으로 설치 확인합니다.



```
vitul)
eicrtle;
3  irutle)
1  ttirrnnivel peenilel(:)
7  ikvrrntie
1  tvnrirnstin)
2  cnreatiet;eatgrent;eronlie (:)
7  cnrienieft)
2  vinonivel peentlel(:)
3  cl)'
inritirnvirenent)
```

Django의 특징

- Django란? - Python 기반의 강력한 웹 프레임워크
- 빠른 개발과 보안성이 강점
- Instagram, Spotify 등에서 사용
- 발표 목표: Django의 개념과 핵심 기능 이해
- 기본적인 Django 웹 서버 구축 과정 학습

Django의 특징

- 빠른 개발: DRY(Don't Repeat Yourself) 원칙
- 보안: SQL Injection, CSRF, XSS 공격 방어
- 확장성: 대규모 트래픽 처리 가능 (ex: Instagram)
- 유지보수성: MVT 패턴 적용, 코드 관리 용이

Django 개발 환경 설정

- Python 및 가상 환경 설정
- Django 설치 및 프로젝트 생성
- 서버 실행 및 개발 환경 확인

예제 코드:

- `python -m venv myenv`
- `source myenv/bin/activate` # (Windows: `myenv\Scripts\activate`)
- `pip install django`
- `django-admin startproject myproject`
- `cd myproject`
- `python manage.py runserver`

Django 프로젝트 구조

- 프로젝트 및 앱 구조 설명
- settings.py, urls.py, views.py 등 주요 파일 소개
- Django 앱 구조 이해

예제 코드:

```
myproject/  
├── manage.py  
├── myproject/  
│   ├── settings.py  
│   ├── urls.py  
│   └── wsgi.py  
└── myapp/  
    ├── models.py  
    ├── views.py  
    └── templates/
```

모델 (Models)

- Django ORM 개념
- 모델 정의 및 마이그레이션
- 예제: **User** 모델 생성 및 데이터베이스 반영

예제 코드:

```
from django.db import models
```

```
class User(models.Model):
```

```
    username = models.CharField(max_length=100)
```

```
    email = models.EmailField(unique=True)
```

```
    joined_at = models.DateTimeField(auto_now_add=True)
```

```
# 마이그레이션 실행
```

```
python manage.py makemigrations
```

```
python manage.py migrate
```

뷰 (Views)

- 함수형 뷰 vs 클래스 기반 뷰
- HTTP 요청 처리 방식
- 예제: 간단한 'Hello World' 뷰 작성

예제 코드:

```
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("Hello, Django!")
```

템플릿 (Templates)

- Django 템플릿 엔진 개요
- 템플릿 문법 및 활용
- 예제: 사용자 맞춤형 템플릿 렌더링

예제 코드:

```
<!-- home.html -->  
<h1>안녕하세요, {{ username }} 님!</h1>
```

```
# views.py에서 렌더링  
from django.shortcuts import render  
def home(request):  
    return render(request, 'home.html', {'username': 'Alice'})
```

URL 라우팅

- URLconf 개념
- URL 매핑 및 동적 URL 처리
- 예제: URL과 뷰 연결하기

예제 코드:

```
from django.urls import path
from .views import hello_world

urlpatterns = [
    path('hello/', hello_world),
]
```


폼과 사용자 입력 처리

- Django Form 클래스
- 폼 검증 및 사용자 입력 처리
- 예제: 회원가입 폼 만들기

예제 코드:

```
from django import forms
```

```
class SignUpForm(forms.Form):
```

```
    username = forms.CharField(max_length=100)
```

```
    email = forms.EmailField()
```

Django 관리자 패널

- 기본 관리자 페이지 설정
- 관리자 계정 생성 및 모델 등록
- 관리자 패널 커스터마이징

예제 코드:

```
# 관리자 계정 생성
```

```
python manage.py createsuperuser
```

```
# 모델 등록
```

```
from django.contrib import admin
```

```
from .models import User
```

```
admin.site.register(User)
```

Django 보안 기능

- CSRF, XSS 방지
- 인증 및 권한 관리
- 예제: 로그인 및 사용자 인증 구현

예제 코드:

```
from django.contrib.auth import authenticate
```

```
user = authenticate(username='testuser', password='password123')
```

Django 웹 서버 배포

- 배포 환경 선택: Heroku, AWS, Nginx + Gunicorn 등
- Gunicorn과 Nginx를 활용한 배포
- 배포 후 유지보수 전략

예제 코드:

- # Gunicorn 실행
- `pip install gunicorn`
- `gunicorn myproject.wsgi:application --bind 0.0.0.0:8000`

성적 평가 :성취기반평가

- 항목: 점수
- 중간 퀴즈 (실기 50 포함) : 100
- 기말 퀴즈 (실기 50 포함) : 100
- 과제 1 : 10
- 과제 2 : 10
- 과제 3 : 10
- 과제 4 : 10
- AD프로젝트 : 50
- 출석 : 10
- 수업참여도 (크리에이터 경진대회) 출품 : 10
- 총점 : 300

- 등급 : 점수
- A+: 245
- A : 210
- B+: 190
- B : 160
- C+: 130
- C : 110
- D+ : 95
- D : 80
- F : 80 이하

첫 Django 프로젝트 생성

프로젝트 생성 명령어 실행

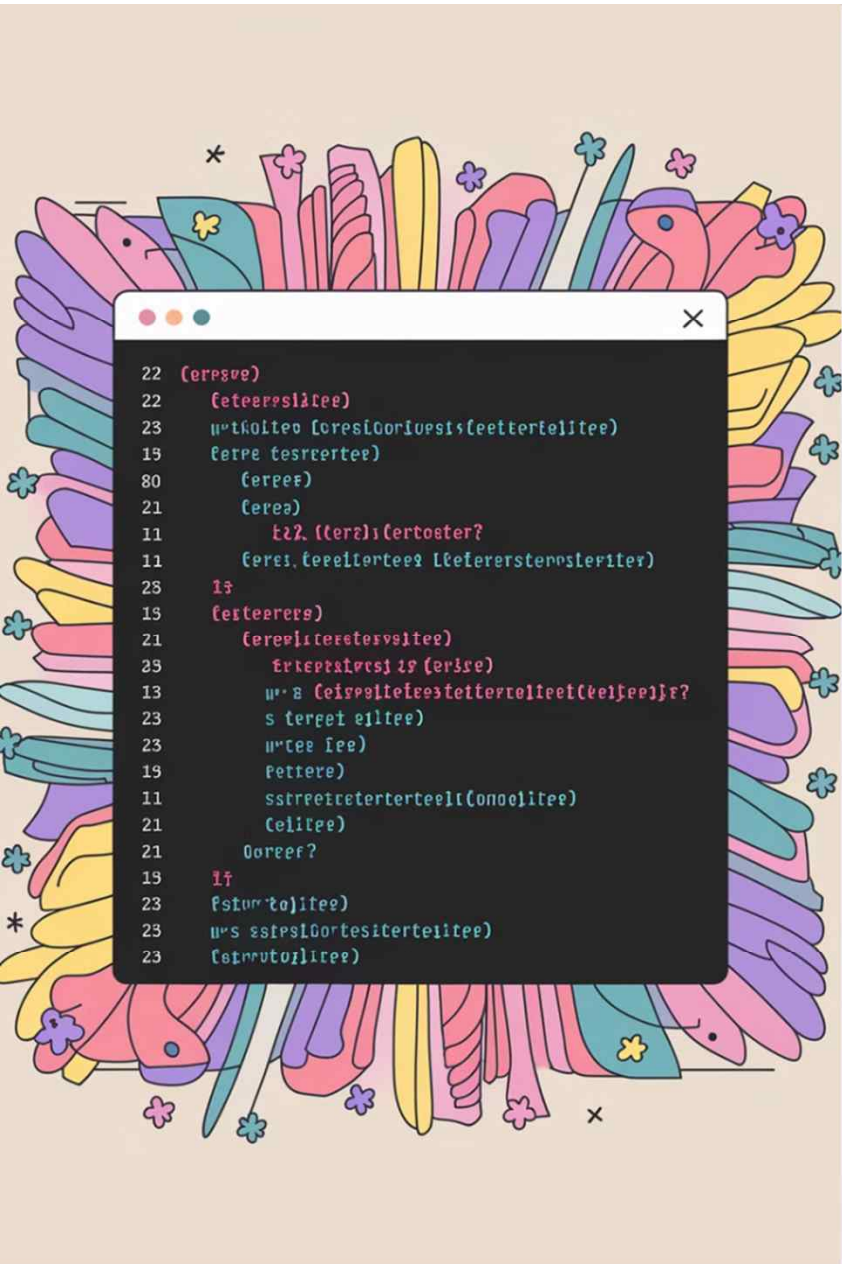
django-admin startproject mysite 명령으로 새 프로젝트를 만듭니다.

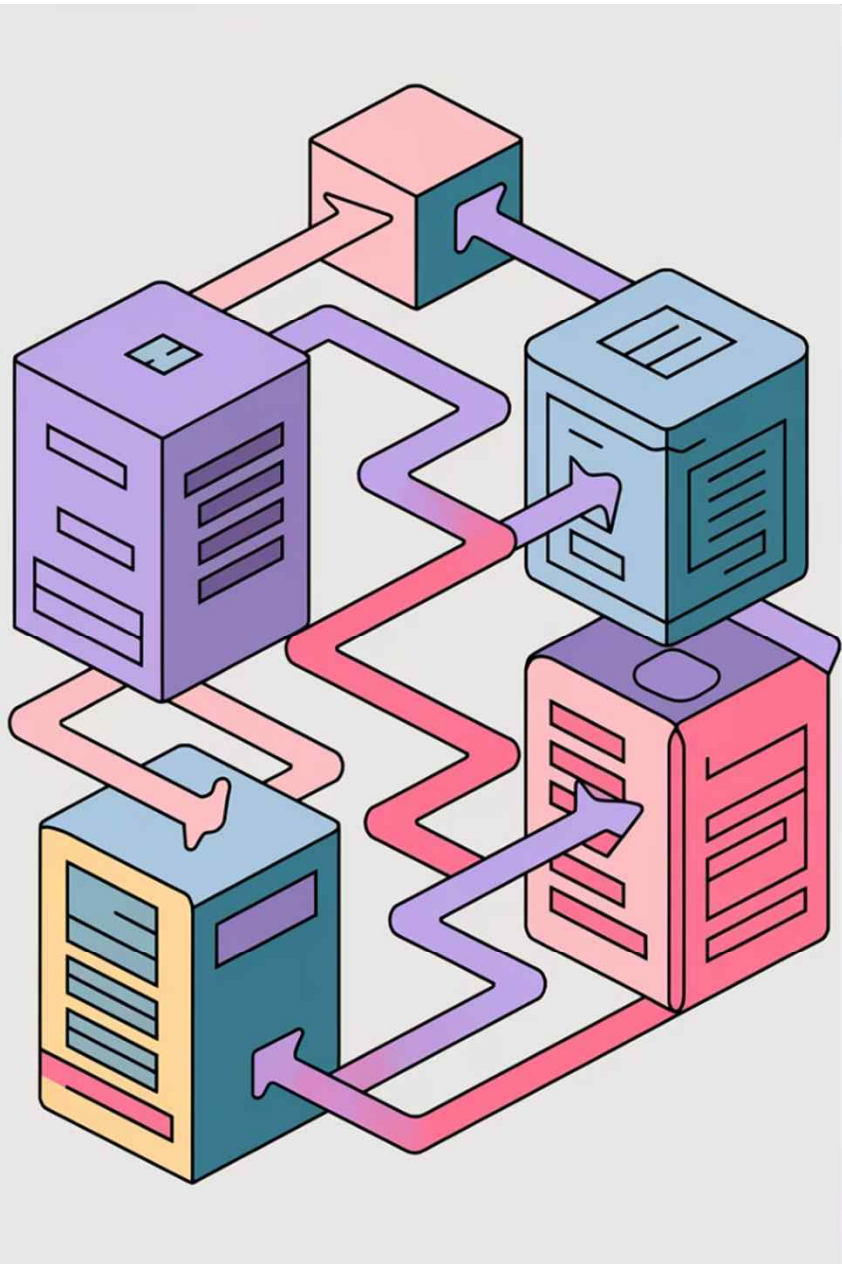
디렉토리 구조 확인

manage.py, settings.py 등 주요 파일들이 생성됩니다.

서버 실행

python manage.py runserver 명령으로 개발 서버를 시작합니다.





Django 애플리케이션 생성

앱 생성 명령어

`python manage.py startapp my app` 명령으로 앱을 생성합니다.

앱 등록

`settings.py`의 `INSTALLED_APPS` 리스트에 새 앱을 추가합니다.

앱 구조

`models.py`, `views.py`, `urls.py` 등의 핵심 파일이 포함됩니다.

settings.py 파일 설정

데이터베이스 설정

DATABASES 딕셔너리에서 데이터베이스 종류와 연결 정보를 설정합니다.

정적 파일 설정

STATIC_URL, STATIC_ROOT를 설정해 정적 파일 경로를 지정합니다.

시간대 설정

TIME_ZONE 변수를 'Asia/Seoul'로 설정해 한국 시간을 사용합니다.

URL 설정하기



urls.py 파일 구조

프로젝트와 앱 수준의 URL 패턴을 정의하는 파일입니다.



URL 패턴 매핑

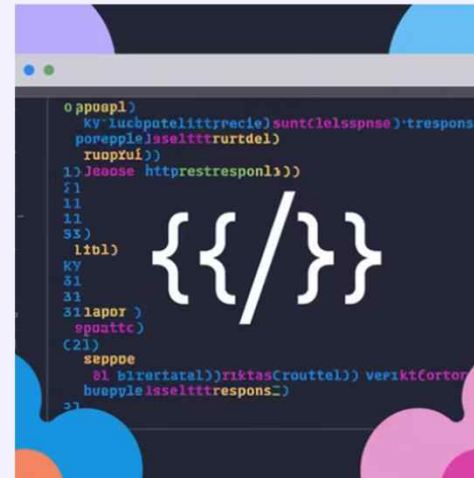
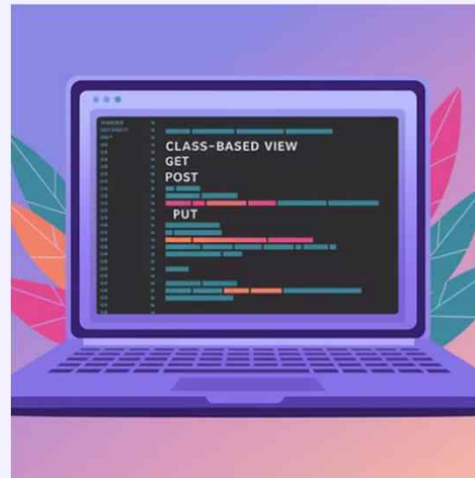
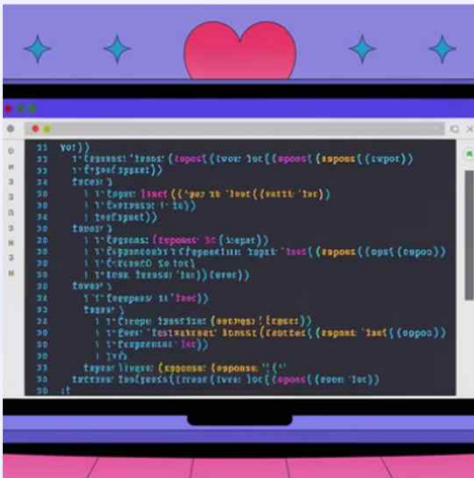
`path()` 함수로 URL을 뷰 함수에 연결합니다.



동적 URL 매개변수

꺾쇠 괄호를 사용해 URL에서 변수를 추출할 수 있습니다.

Views 작성하기



1 함수 기반 뷰

request 객체를 받아 처리하는 간단한 Python 함수입니다.

2 클래스 기반 뷰

코드 재사용과 확장이 용이한 객체지향적 접근법입니다.

3 응답 객체

HttpResponse를 반환하여 클라이언트에게 데이터를 전송합니다.

템플릿 사용하기

템플릿 언어

HTML에 동적 기능을 추가하는 특별한 문법입니다.

템플릿 상속

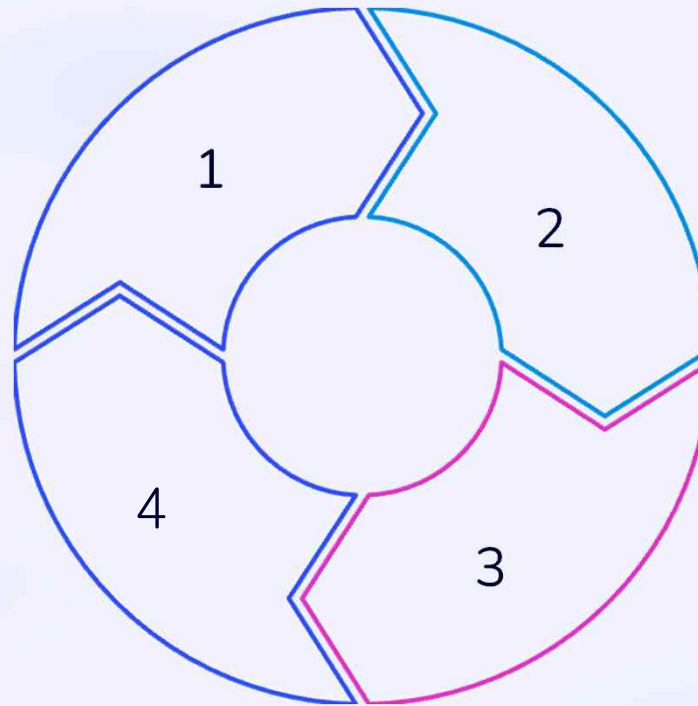
{% extends %} 태그로 기본 템플릿을 확장할 수 있습니다.

변수와 필터

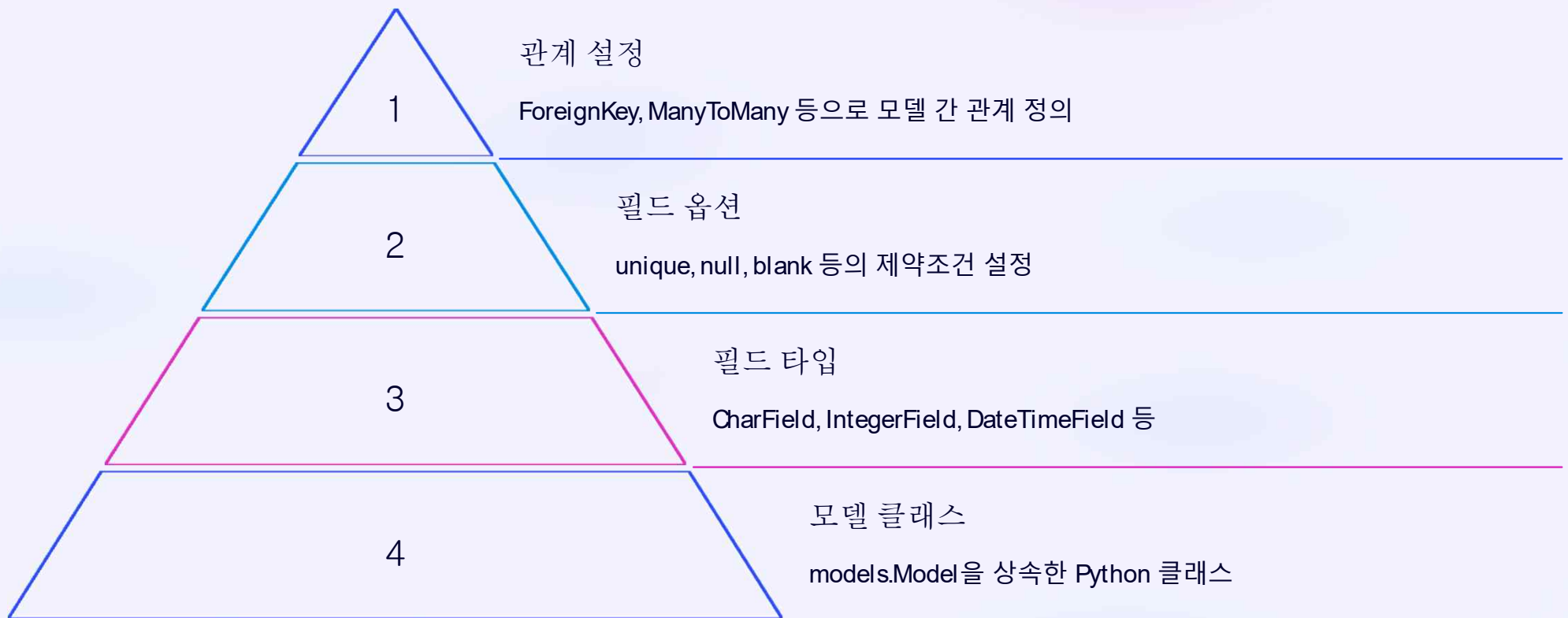
{{ variable|filter }} 형태로 데이터를 표시하고 변환합니다.

템플릿 태그

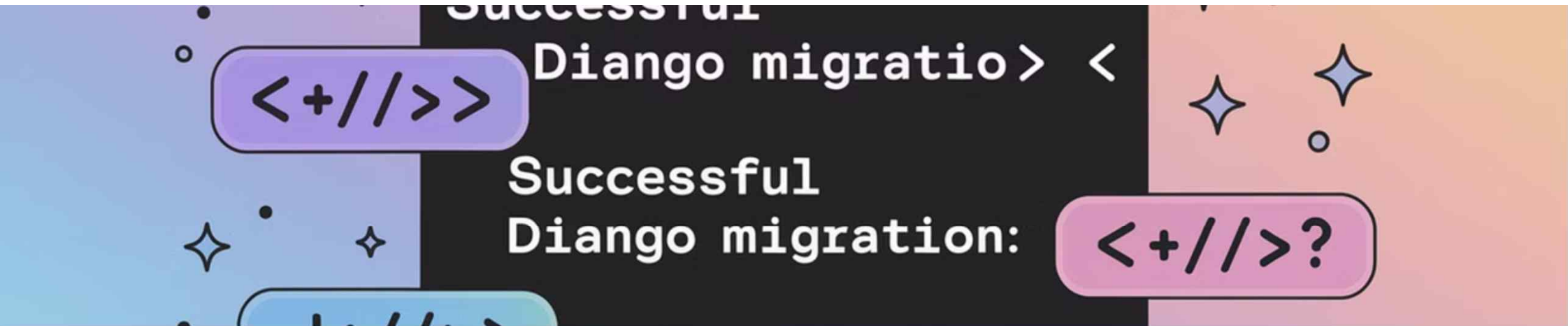
{% tag %} 형태로 제어 흐름과 논리를 구현합니다.



모델 정의하기



모델은 데이터베이스 테이블의 구조를 Python 코드로 정의합니다. 각 모델 클래스는 데이터베이스의 테이블에 매핑됩니다.



데이터베이스 마이그레이션

마이그레이션 생성

python manage.py makemigrations 명령으로 변경사항을 감지합니다.

마이그레이션 적용

python manage.py migrate 명령으로 데이터베이스에 변경을 적용합니다.

마이그레이션 확인

python manage.py showmigrations로 상태를 확인합니다.



Django ORM 사용하기



쿼리셋 API

모델.objects.all()로 모든 객체를 가져오거나 get(), filter()로 특정 객체를 선택합니다.



필터링과 정렬

filter(condition).order_by('field')로 조건부 검색과 정렬을 수행합니다.



집계 함수

count(), sum(), avg() 등의 함수로 데이터를 집계합니다.



Django 관리자 인터페이스

1

관리자 계정 생성

`python manage.py createsuperuser` 명령으로 계정을 만듭니다.

2

모델 등록

`admin.py` 파일에 `admin.site.register(Model)`로 모델을 등록합니다.

3

커스터마이징

`ModelAdmin` 클래스로 관리자 페이지의 모습과 기능을 변경합니다.

인증과 권한

1

권한 관리

세부적인 접근 제어

2

그룹 설정

유사한 권한 집합

3

사용자 인증

로그인/로그아웃 기능

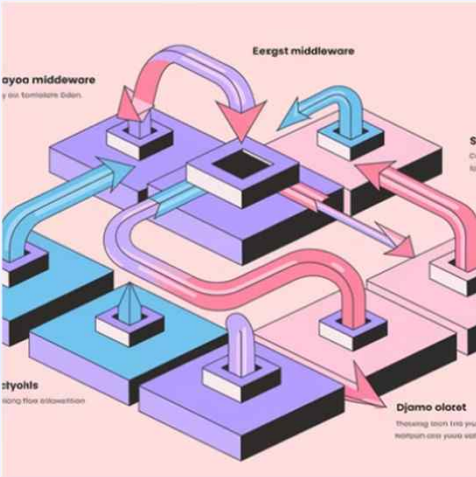
4

User 모델

기본 사용자 정보

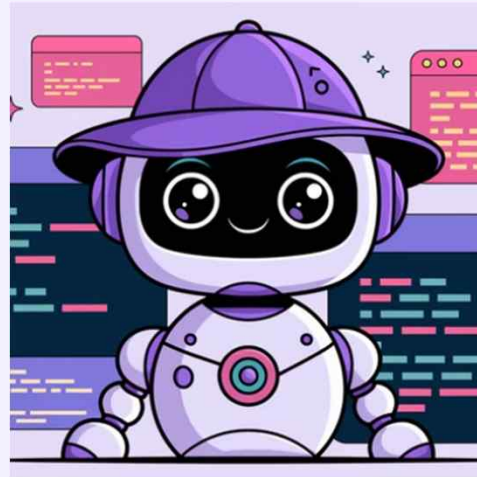
Django의 인증 시스템은 사용자 관리, 그룹 설정, 권한 제어를 통합적으로 제공합니다. auth 모듈은 안전한 사용자 인증을 쉽게 구현할 수 있도록 돕습니다.

미들웨어 사용하기



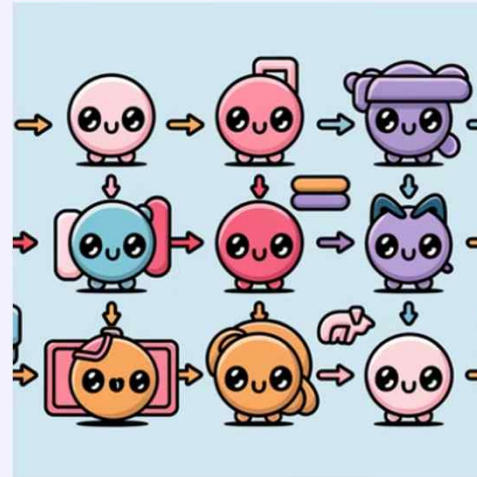
기본 미들웨어

Django는 보안, 세션, CSRF 보호 등을 위한 기본 미들웨어를 제공합니다.



커스텀 미들웨어

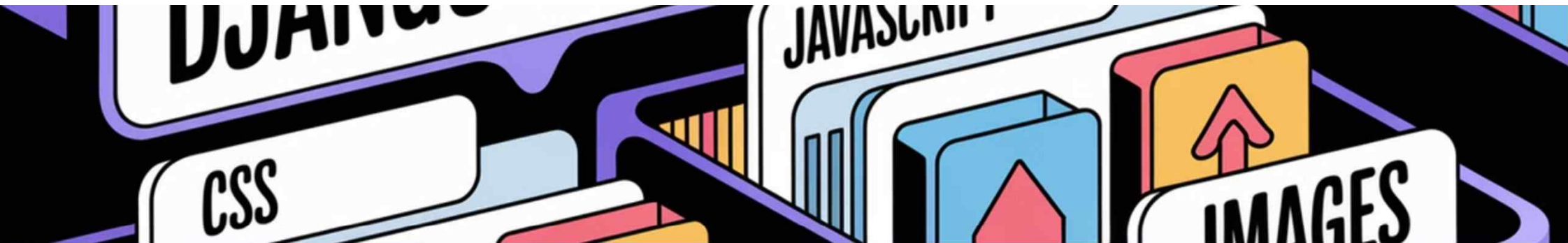
`process_request`와 `process_response` 메서드를 구현하여 요청/응답을 처리합니다.



순서의 중요성

미들웨어 실행 순서는 `settings.py`의 `MIDDLEWARE` 리스트에 따라 결정됩니다.





정적 파일 관리

정적 파일 구성

CSS, JavaScript, 이미지 등의 파일을 앱의 static 폴더에 저장합니다.

1

설정 구성

STATIC_URL, STATICFILES_DIRS, STATIC_ROOT를 settings.py에 설정합니다.

2

파일 수집

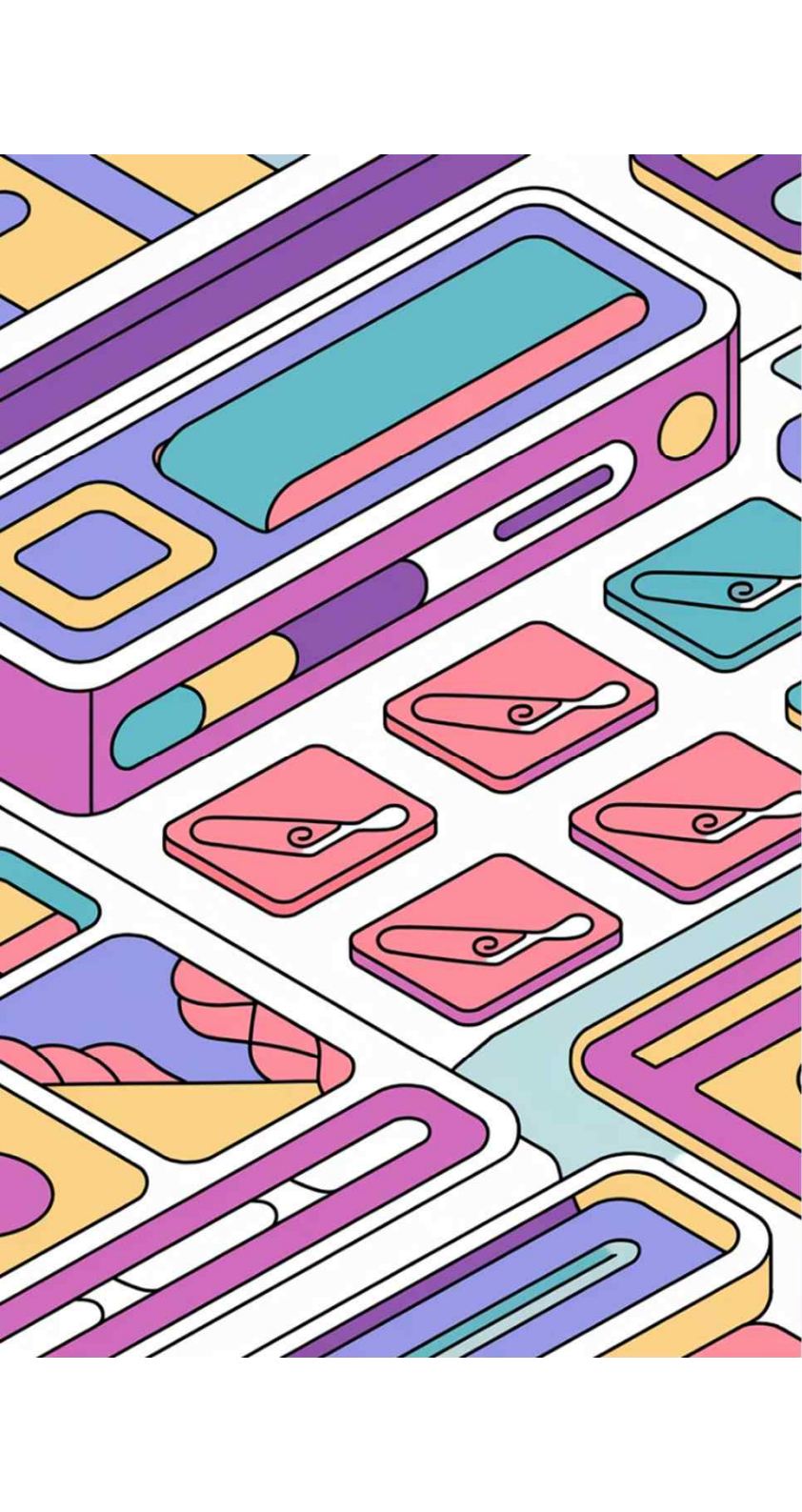
python manage.py collectstatic으로 정적 파일을 한 곳에 모읍니다.

3

템플릿에서 사용

{% load static %}와 {% static 'path' %}로 정적 파일을 참조합니다.

4



미디어 파일 다루기

- 1 미디어 설정
settings.py에 MEDIA_URL과 MEDIA_ROOT를 정의합니다.
- 2 파일 필드 사용
모델에 FileField나 ImageField를 추가해 파일을 저장합니다.
- 3 업로드 처리
request.FILES를 통해 폼에서 전송된 파일을 처리합니다.
- 4 파일 서빙
개발 중에는 urls.py에 static() 함수로 미디어 파일을 서빙합니다.

Django REST framework 소개



REST 구조

리소스 중심의 API 구조를 쉽게 구현할 수 있습니다.



Serializer

모델 데이터를 JSON 등의 형식으로 변환하거나 역변환합니다.



ViewSet

CRUD 작업을 위한 일관된 인터페이스를 제공합니다.

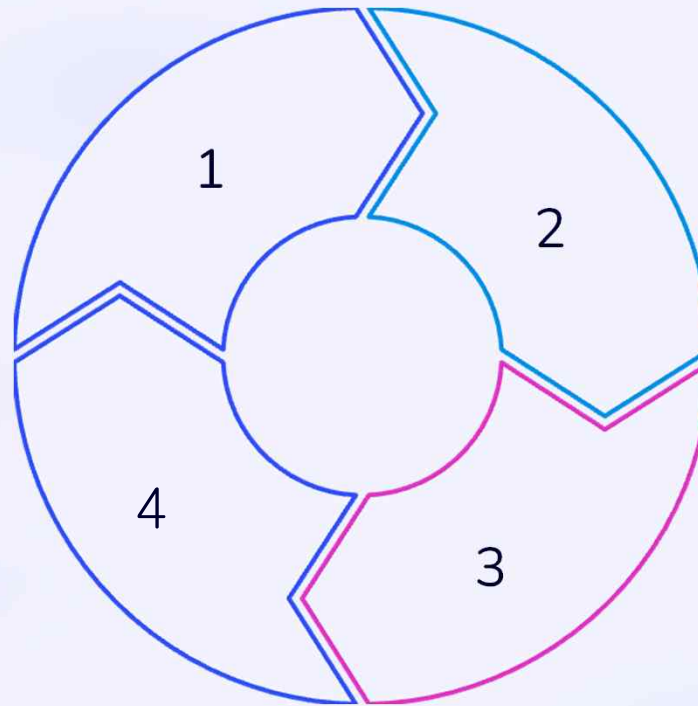
캐싱 구현하기

캐시 백엔드

Memcached, Redis, 데이터베이스 등
다양한 캐시 시스템 지원

저수준 API

cache.set()과 cache.get()으로 세밀한
캐싱 제어



뷰 레벨 캐싱

@cache_page 데코레이터로 전체 뷰
결과를 캐싱

템플릿 캐싱

{% cache %} 태그로 템플릿 일부분만
캐싱

테스팅

단위 테스트

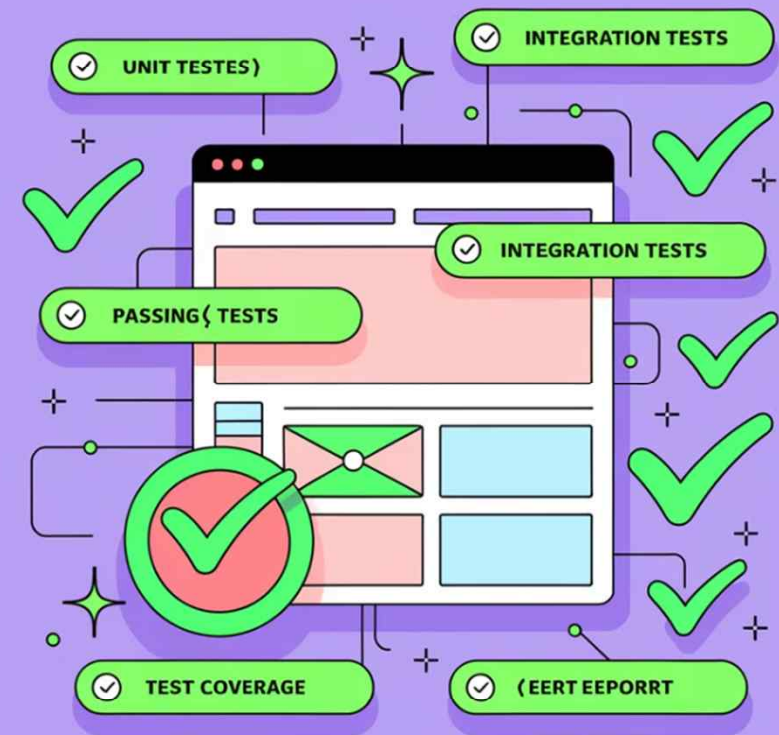
TestCase 클래스를 상속해 모델, 뷰, 폼 등의 개별 구성 요소를 테스트합니다.

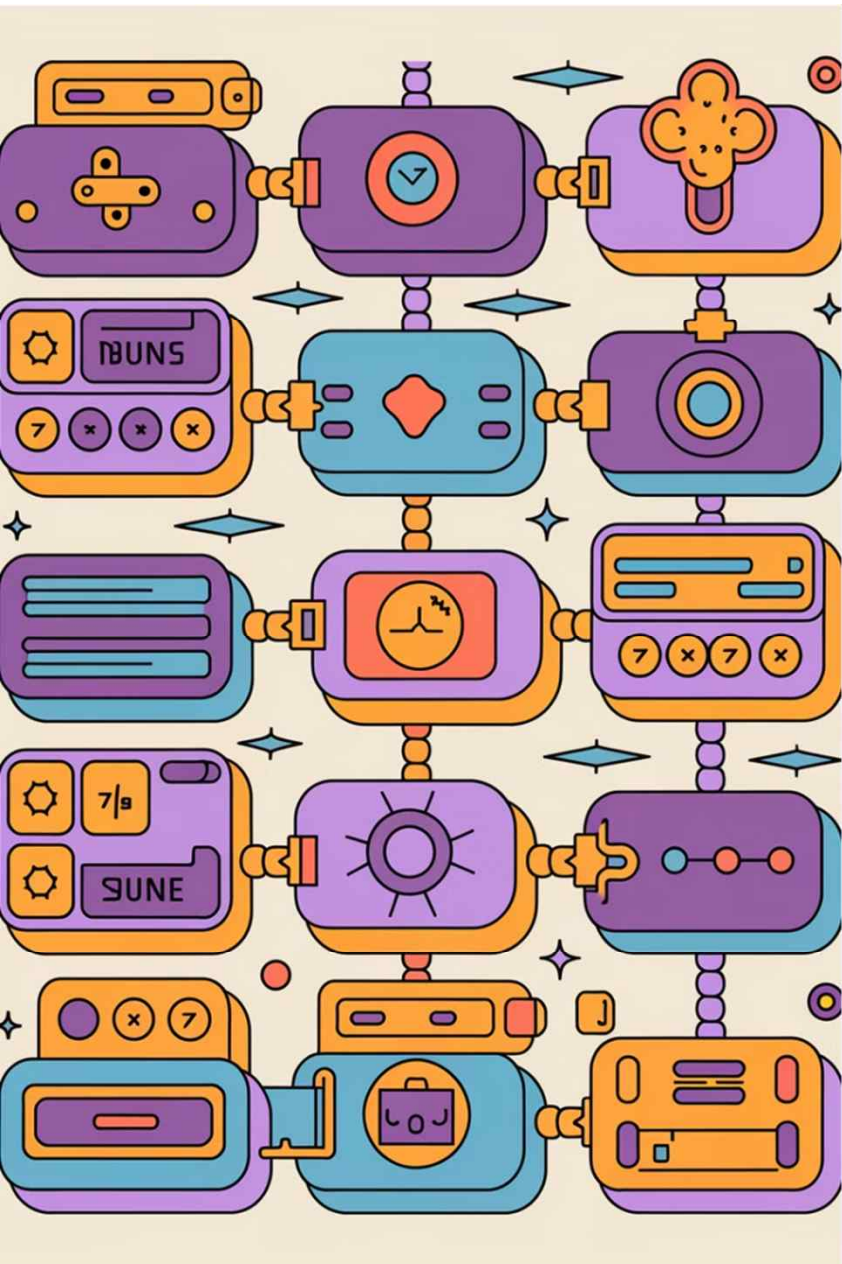
테스트 클라이언트

HTTP 요청을 시뮬레이션하여 뷰와 URL 패턴을 테스트합니다.

커버리지 측정

coverage 도구로 테스트가 코드베이스를 얼마나 다루는지 측정합니다.





시그널 사용하기

1

내장 시그널

pre_save, post_save와 같은 모델 관련 시그널이 기본 제공됩니다.

2

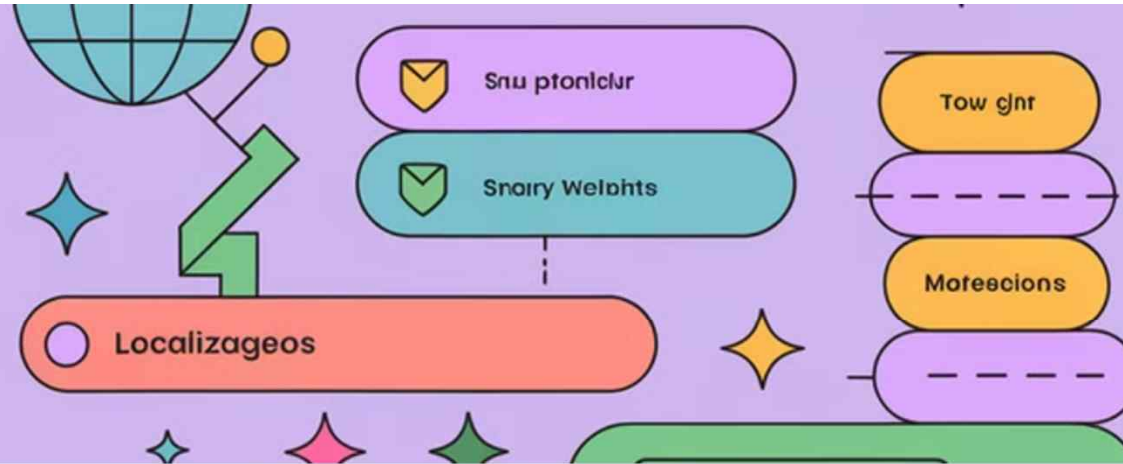
커스텀 시그널

django.dispatch.Signal()로 자신만의 시그널을 정의할 수 있습니다.

3

시그널 연결

@receiver 데코레이터로 시그널에 핸들러 함수를 연결합니다.



Django의 국제화와 지역화



번역 문자열

`gettext_lazy` as `_`를 임포트하고 `_`('텍스트')로 번역 가능한 문자열을 표시합니다.



언어 설정

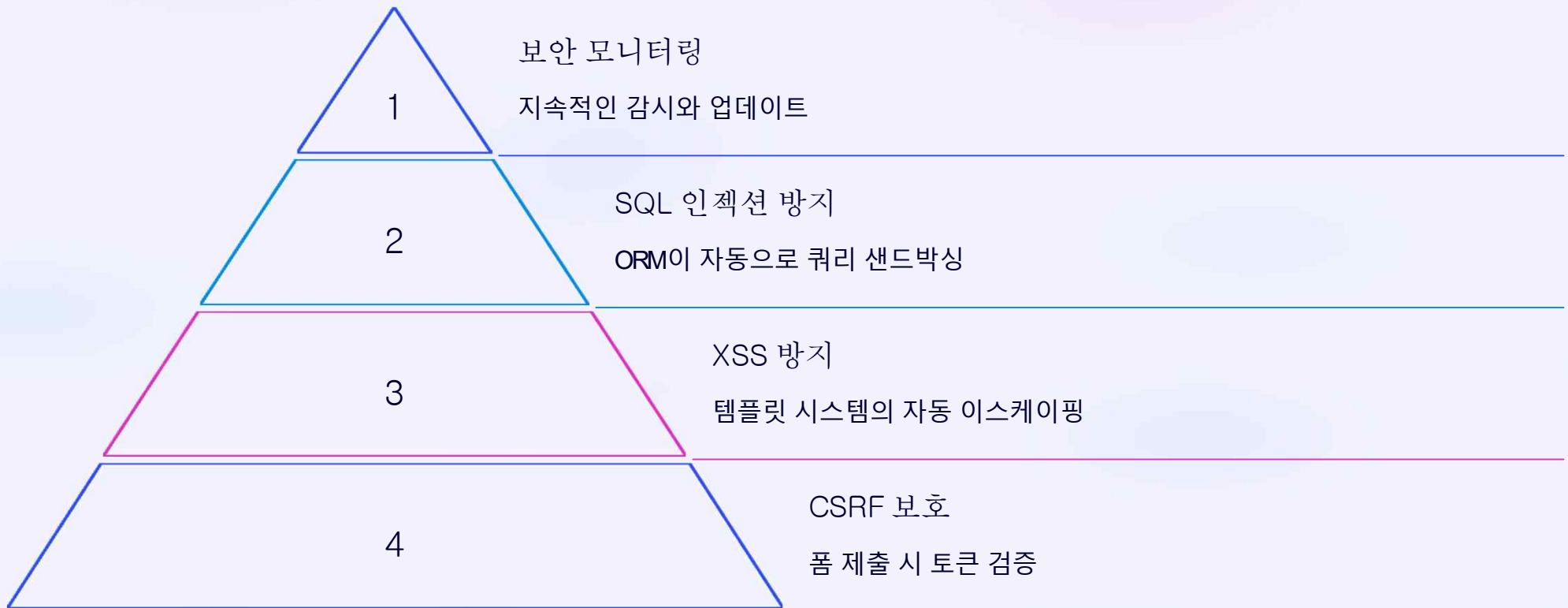
`settings.py`의 `LANGUAGE_CODE`와 `USE_L18N`으로 언어를 설정합니다.



시간대 처리

`USE_TZ=True`로 시간대 인식을 활성화하고 날짜/시간을 처리합니다.

보안 기능



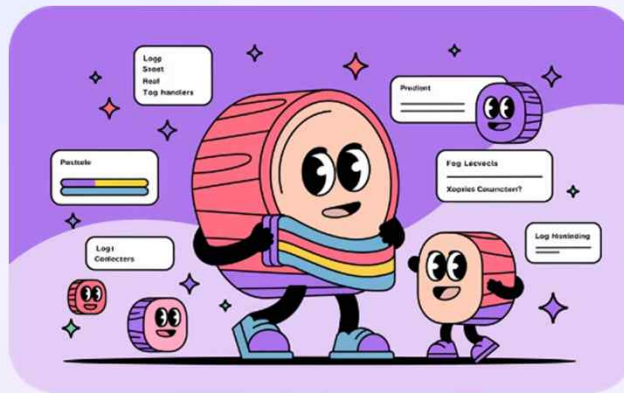
Django는 웹 애플리케이션 보안을 최우선으로 고려합니다. 기본적으로 많은 보안 기능이 활성화되어 있어 개발자가 별도로 구현하지 않아도 안전한 사이트를 구축할 수 있습니다.

Django 디버깅



Django Debug Toolbar

요청 정보, SQL 쿼리, 캐시 사용 등을 웹 페이지에서 직접 확인할 수 있습니다.



로깅 설정

settings.py의 LOGGING 설정으로 다양한 로그 레벨과 핸들러를 구성합니다.



에러 페이지

DEBUG=True일 때 상세한 오류 정보를 제공하는 페이지가 표시됩니다.



성능 최적화

데이터베이스 쿼리 분석

django-debug-toolbar로 N+1 쿼리 문제를 식별합니다.

관계 최적화

`select_related`와 `prefetch_related`로 관련 객체를 효율적으로 가져옵니다.

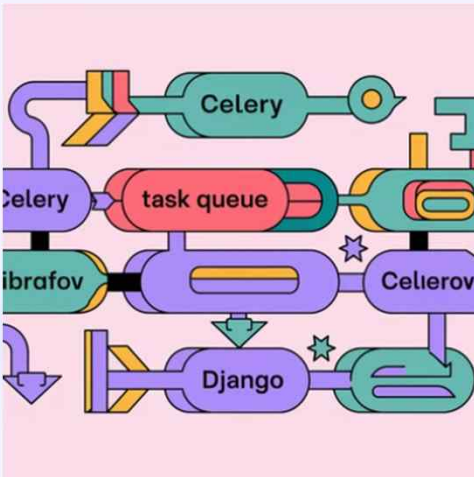
인덱싱 전략

`db_index=True`로 자주 검색하는 필드에 인덱스를 추가합니다.

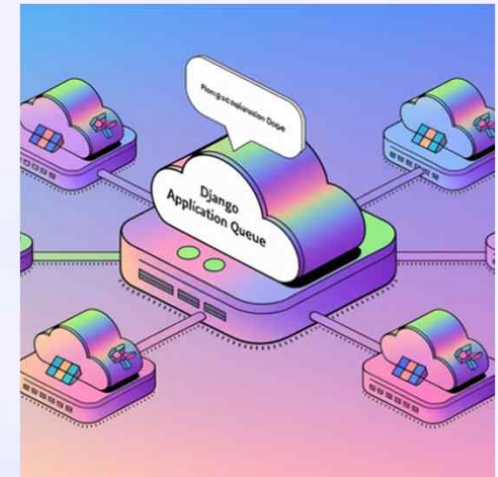
캐싱 적용

자주 접근하는 데이터에 적절한 캐싱 전략을 구현합니다.

Django와 비동기 작업



```
schedule() - asyrfiecdls)
cltiys (yxciceeliing> - (seety
sdui)>|
suiitt) kiortiti) - +ipço-
sdui)>|
suiitt) tiottitkar() (+>()
sdui)>|
suiitt) ticery +
```



- 1 Celery 통합

Django 애플리케이션에 Celery를 연결하여 비동기 작업을 처리합니다.
- 2 태스크 구현

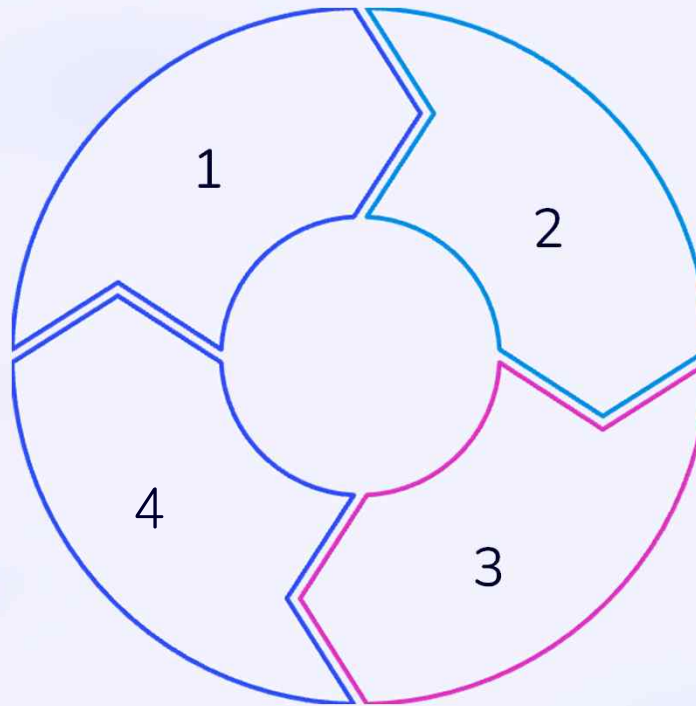
@task 데코레이터로 비동기적으로 실행할 함수를 정의합니다.
- 3 결과 처리

task.delay().get()으로 비동기 작업의 결과를 가져올 수 있습니다.

Django 채널

웹소켓 지원
양방향 통신 프로토콜 구현

그룹 관리
`channel_layer.group_send()`로 다수 클라이언트에 브로드캐스팅



채널 레이어

Redis, RabbitMQ 등의 백엔드로 메시지 전달

비동기 소비자

`WebsocketConsumer` 클래스로 실시간 이벤트 처리

Django와 GraphQL

Graphene-Django

Django 모델을 GraphQL 스키마로 변환하는 라이브러리입니다.

스키마 정의

모델 필드에 기반한 타입과 쿼리를 정의합니다.

쿼리 및 뮤테이션

데이터 조회와 수정을 위한 GraphQL 작업을 구현합니다.



Django 확장 기능

1000+

PyPI 패키지

Django 확장 기능을 제공하는 서드
파티 패키지가 1000개 이상 존재합
니다.

250K+

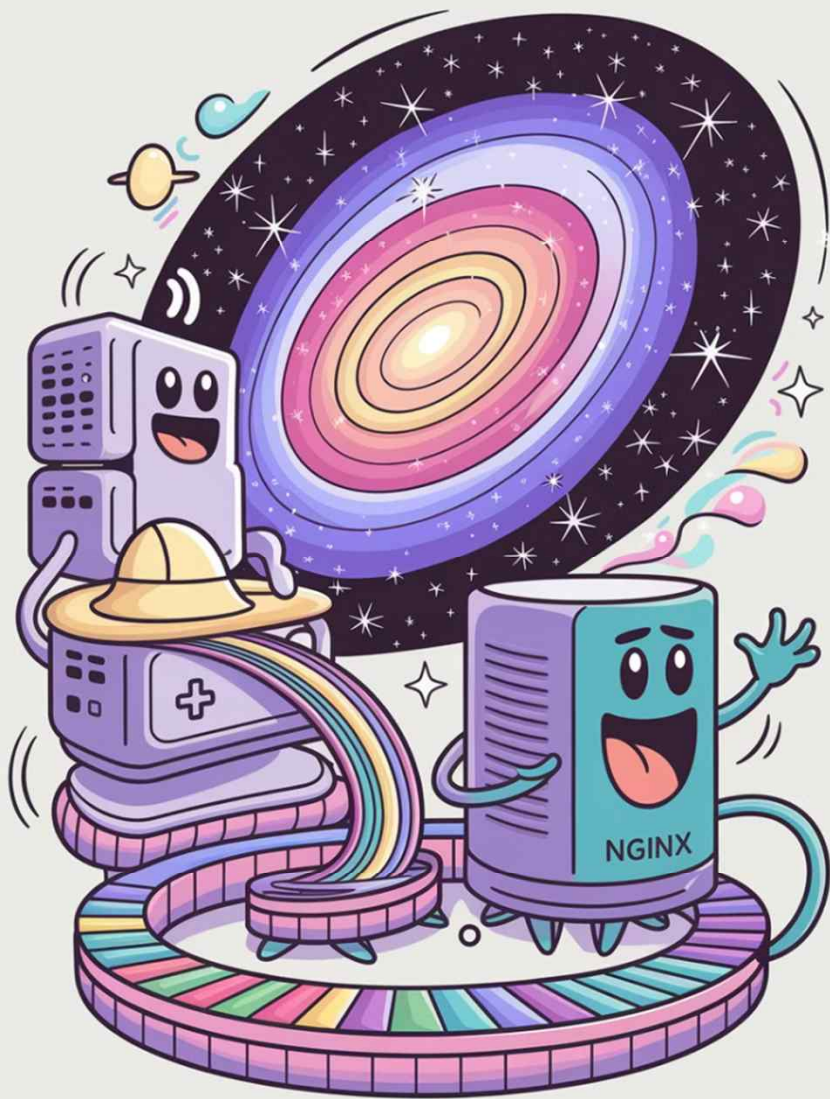
다운로드

인기 패키지는 월 25만 건 이상의 다
운로드를 기록합니다.

5+

주요 분야

인증, 폼, 관리자, REST API, 결제 등
다양한 영역을 다룹니다.



Django 배포하기

1

WSGI 서버 설정

Gunicorn이나 uWSGI와 같은 WSGI 서버를 구성합니다.

2

웹 서버 연결

Nginx나 Apache를 프록시로 설정하여 정적 파일을 효율적으로 제공합니다.

3

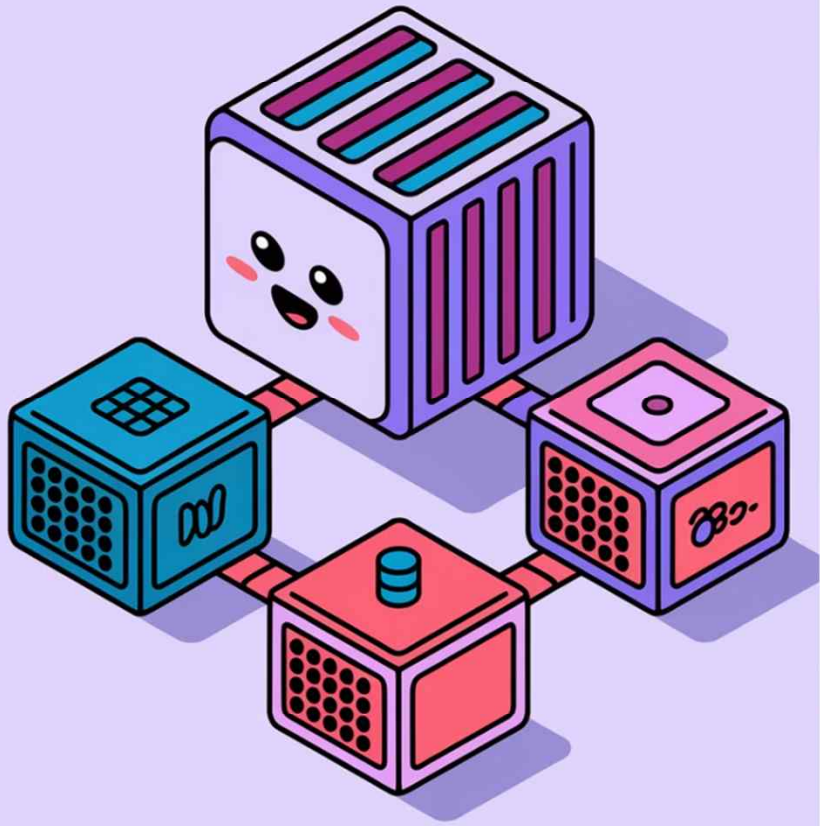
환경 변수 설정

비밀 키와 데이터베이스 정보를 환경 변수로 관리합니다.

4

데이터베이스 마이그레이션

production 환경에서 migrate 명령을 실행하여 데이터베이스를 준비합니다.



Django와 Docker

Dockerfile 작성

Python 이미지를 기반으로 Django 애플리케이션 환경을 구성합니다.

docker-compose.yml

웹, 데이터베이스, 캐시 등 여러 서비스를 정의하고 연결합니다.

볼륨 설정

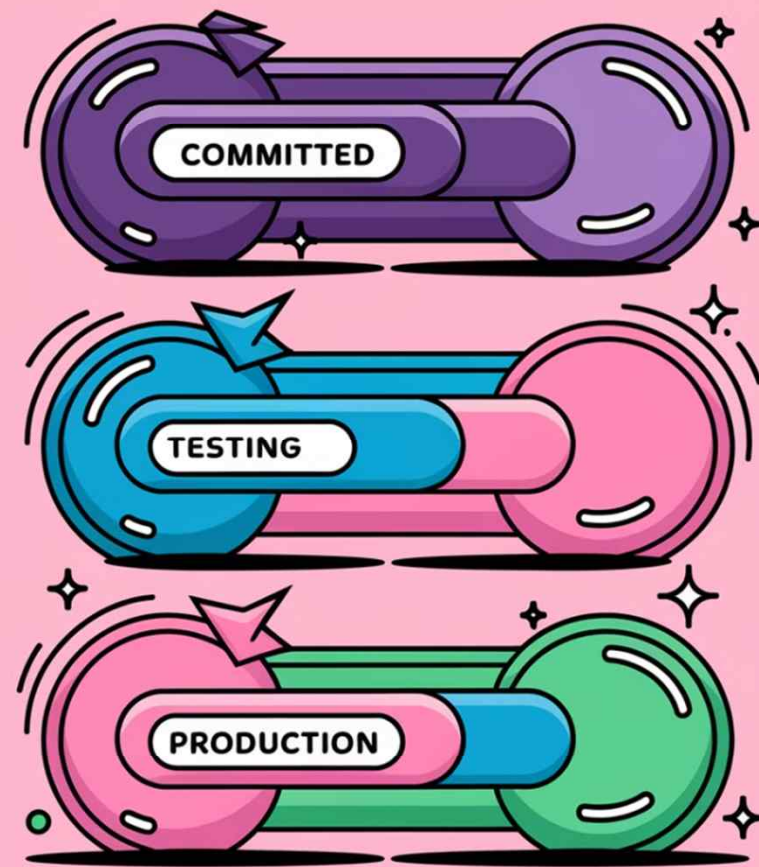
코드와 데이터를 컨테이너와 호스트 간에 공유합니다.

컨테이너 실행

docker-compose up 명령으로 전체 환경을 한 번에 시작합니다.

CI/CD 파이프라인 구축

- 1 코드 푸시
개발자가 GtHub 저장소에 코드를 푸시합니다.
- 2 자동 테스트
GtHub Actions가 테스트 스위트를 실행합니다.
- 3 빌드 과정
테스트 통과 시 Docker 이미지를 빌드합니다.
- 4 자동 배포
완성된 이미지가 프로덕션 환경에 배포됩니다.



Django 모니터링



Sentry 통합

실시간 오류 추적과
알림을 제공하여 문제
를 신속하게 파악합니
다.



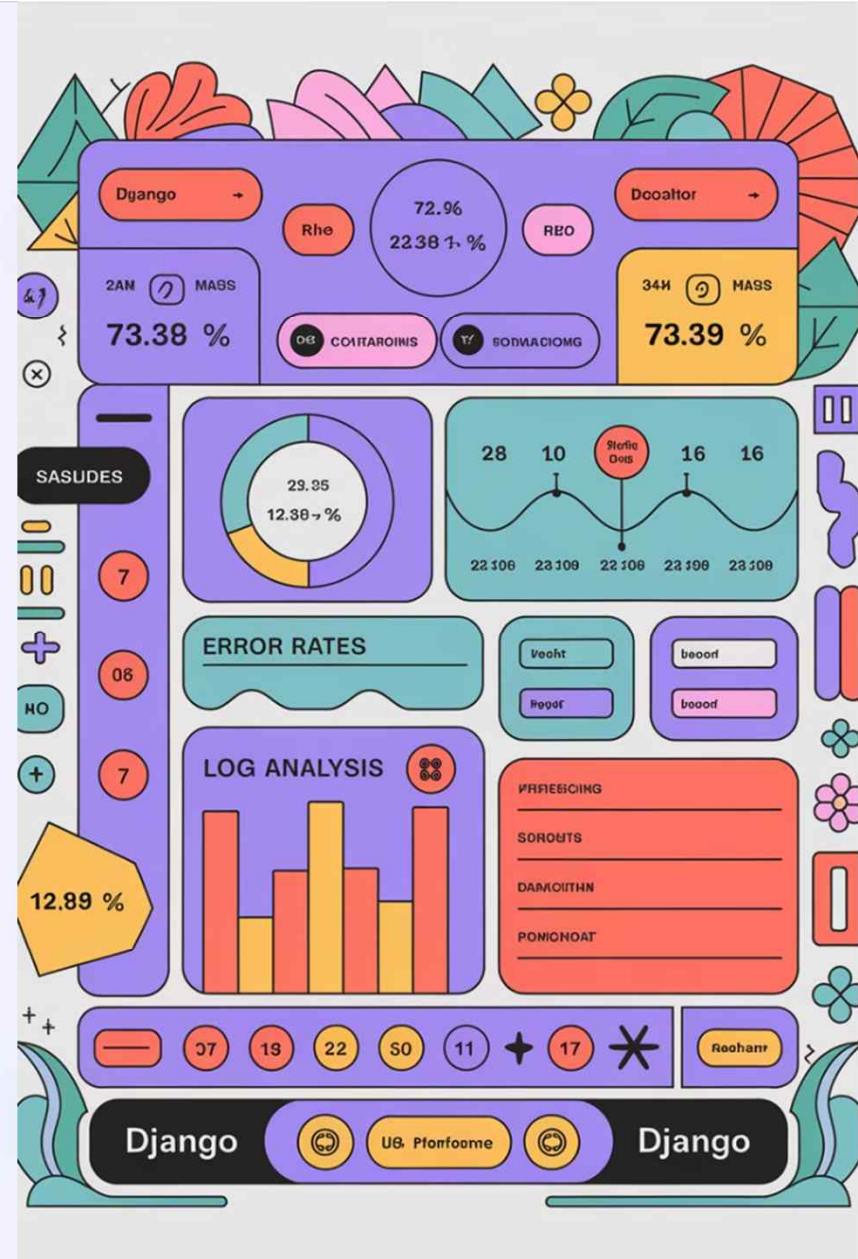
로그 분석

ELK 스택이나 Graylog
로 로그를 집계하고
시각화합니다.



성능 모니터링

New Relic이나 Datadog으로 응답 시간과 리
소스 사용량을 추적합
니다.



Django와 AWS



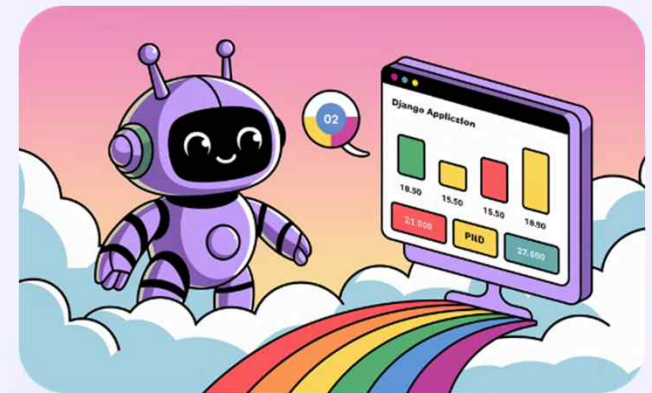
Elastic Beanstalk

인프라 관리 없이 Django 애플리케이션을 쉽게 배포할 수 있습니다.



S3 스토리지

정적 파일과 미디어 파일을 확장성 있게 저장하고 제공합니다.



RDS 데이터베이스

관리형 PostgreSQL이나 MySQL 데이터베이스를 Django와 연결합니다.

Django 프로젝트 구조화

1

재사용 가능한 앱

독립적으로 작동하는 패키지형 앱

2

설정 파일 분리

개발/테스트/프로덕션 환경별 설정

3

논리적 앱 분할

기능과 역할에 따른 앱 구성

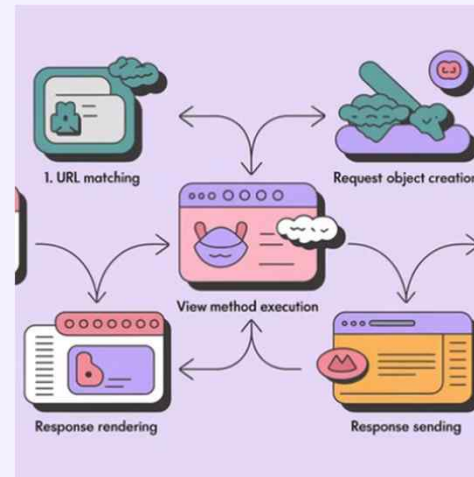
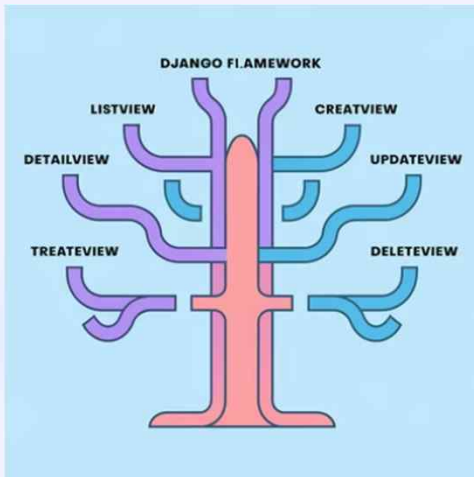
4

기본 구조 설계

프로젝트 골격 계획

효율적인 Django 프로젝트 구조는 유지보수성과 확장성을 크게 향상시킵니다. 기능별로 앱을 분리하고, 환경별로 설정을 구분하는 것이 중요합니다.

Django의 클래스 기반 뷰 심화



- 1 제네릭 뷰
ListView, DetailView 등 일반적인 패턴을 구현한 뷰 클래스를 활용합니다.
- 2 Mixin 활용
LoginRequiredMixin 등의 믹스인으로 공통 기능을 조합합니다.
- 3 커스텀 클래스 뷰
기존 뷰를 상속하고 메서드를 오버라이드하여 기능을 확장합니다.

Django 템플릿 태그와 필터

커스텀 템플릿 태그

@register.simple_tag 데코레이터로 새로운 태그 함수를 정의합니다.

커스텀 필터

@register.filter로 템플릿에서 값을 변환하는 필터를 만듭니다.

인클루전 태그

@register.inclusion_tag로 재사용 가능한 템플릿 블록을 정의합니다.

라이브러리 패키징

templatetags 패키지로 구성하여 여러 프로젝트에서 재사용합니다.



Django와 비관계형 데이터베이스

MongoDB 연동

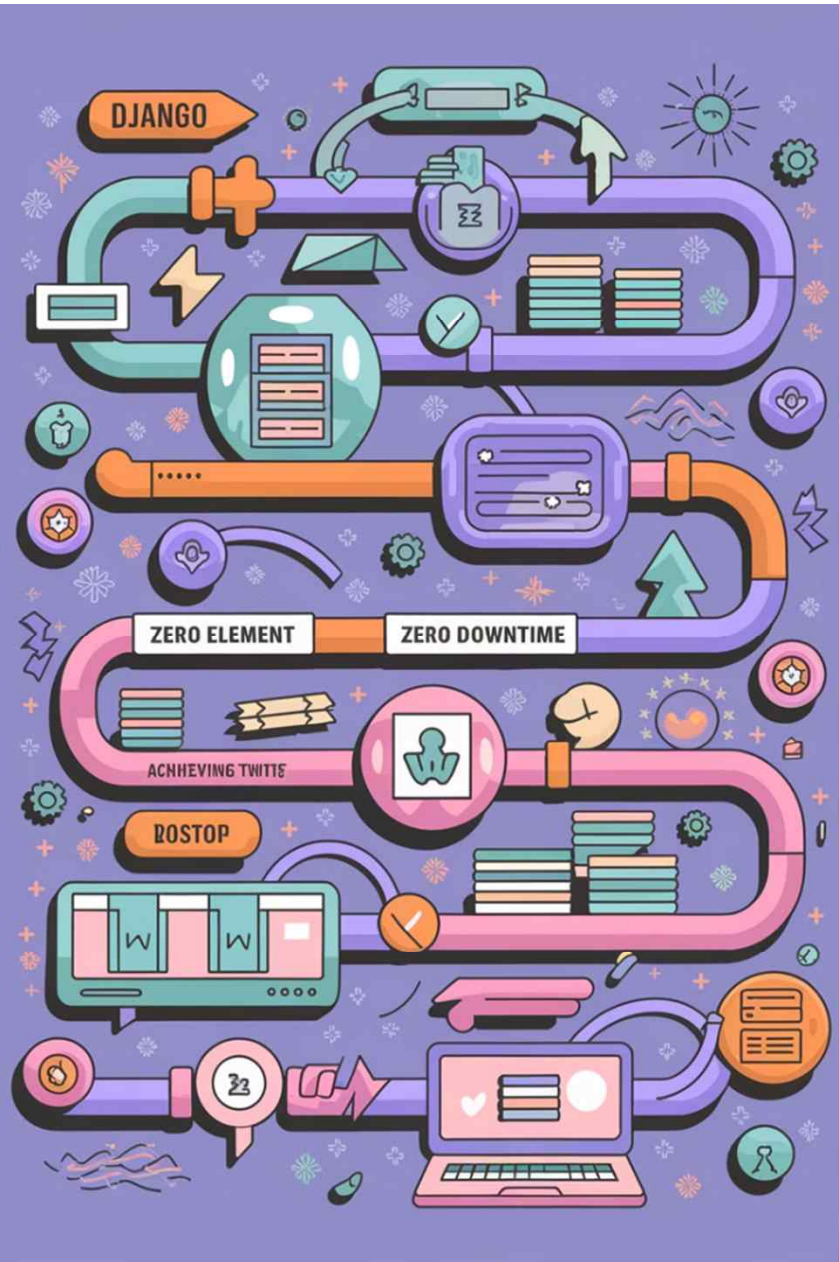
django 패키지를 사용해 Django ORM을 MongoDB에 매핑합니다.

Redis 활용

django-redis로 캐싱, 세션 저장, 메시지 큐 등을 구현합니다.

Elasticsearch

django-elasticsearch-dsl로 강력한 전문 검색 기능을 추가합니다.



Django 마이그레이션 고급 기법

1 데이터 마이그레이션

RunPython을 사용해 스키마 변경과 함께 데이터도 변환합니다.

2 마이그레이션 충돌

migration --merge 명령으로 브랜치 병합 시 발생하는 충돌을 해결합니다.

3 스키마 변경 전략

대규모 테이블 변경 시 무중단 배포를 위한 다단계 마이그레이션을 계획합니다.

4 마이그레이션 스쿼싱

squashmigrations 명령으로 여러 마이그레이션을 하나로 압축합니다.

Django 커스텀 필드 타입

커스텀 모델 필드

`models.Field`를 상속하여 특별한 데이터 타입을 지원하는 필드를 만듭니다.

폼 필드 커스터마이징

`forms.Field`를 확장하여 사용자 입력을 검증하고 변환합니다.

위젯 개발

`forms.widgets.Widget`을 상속하여 HTML 렌더링 방식을 제어합니다.

필드 통합

`deconstruct()` 메서드로 마이그레이션 지원을 구현합니다.



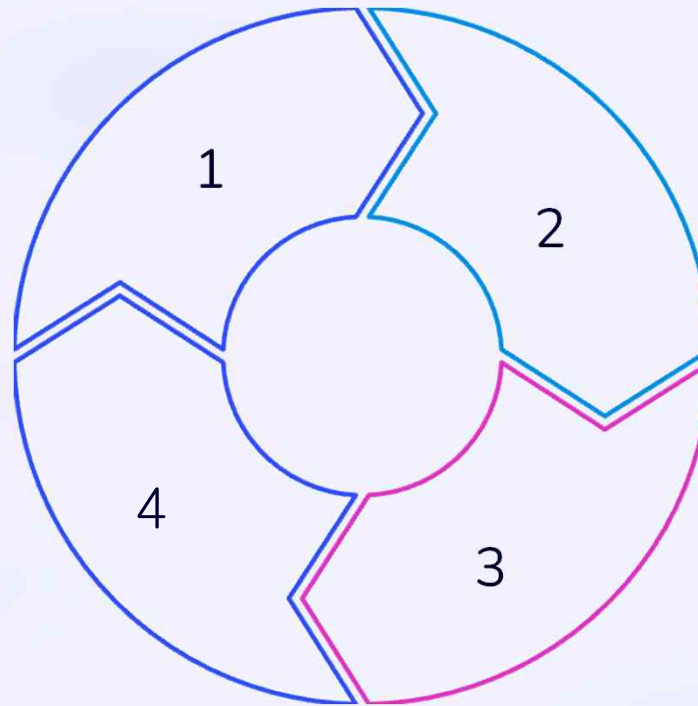
Django와 백그라운드 작업

작업 정의
비동기적으로 실행할 함수를 태스크
로 등록

작업 예약
일정에 따라 자동 실행되는 주기적 태
스크 설정

작업 처리
워커 프로세스가 큐에서 태스크를 가
져와 실행

결과 저장
태스크 실행 결과와 상태 정보 기록





Django 보안 강화



Two-Factor 인증

비밀번호 외에 추가 인증 요소를 도입하여 계정 보안을 강화합니다.



비밀번호 정책

복잡성 요구사항과 주기적 변경을 강제하는 정책을 구현합니다.



API 인증

토큰, JWT, OAuth2를 통한 안전한 API 접근 제어를 설정합니다.

Django 프로파일링

cProfile 사용

`python -m cProfile manage.py runserver`
로 코드 실행 시간을 측정합니다.

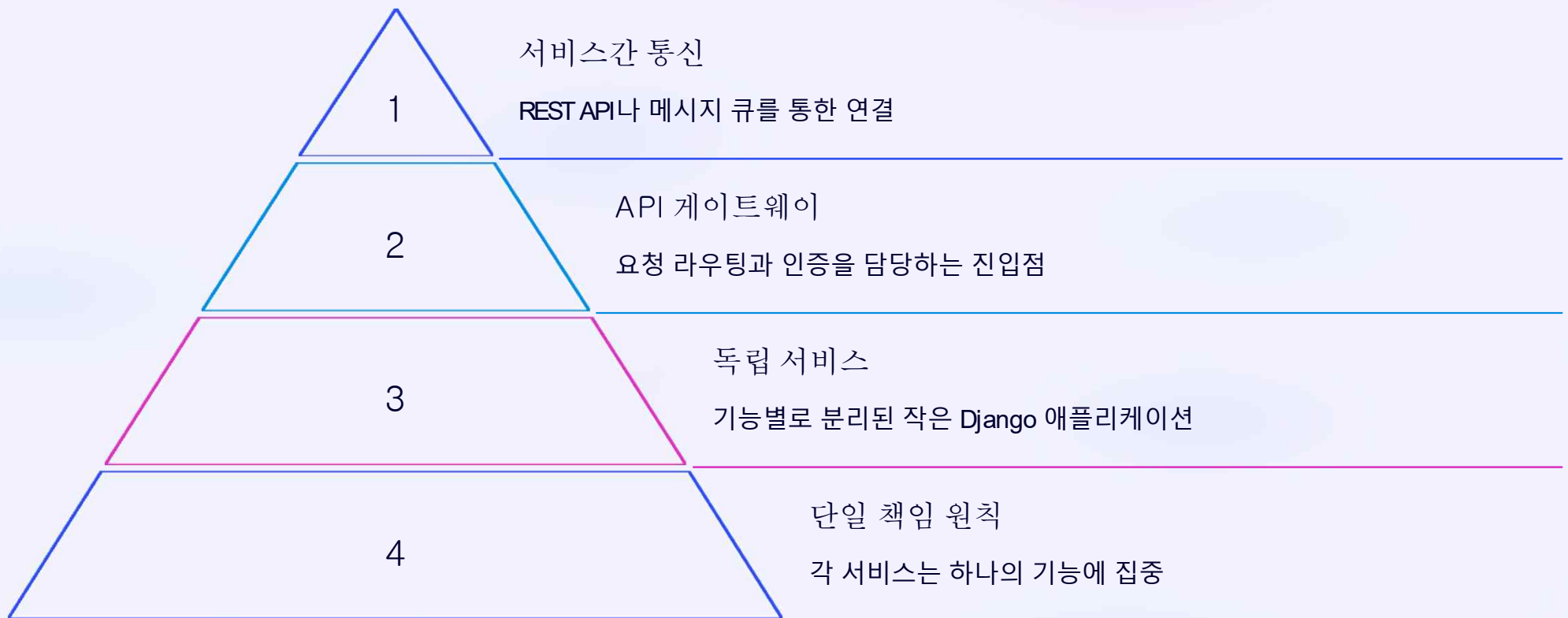
메모리 분석

`memory_profiler` 패키지로 메모리 사용량을 추적하고 누수를 찾습니다.

병목 식별

Django Debug Toolbar와 `Slk`를 통해 느린 쿼리와 뷰를 발견합니다.

Django와 마이크로서비스



마이크로서비스 아키텍처에서 Django는 개별 서비스를 구현하는 데 적합합니다. 각 서비스는 독립적으로 개발, 배포, 확장될 수 있습니다.



Django 업그레이드 전략

변경사항 파악

새 버전의 릴리스 노트를 통해
주요 변경점을 확인합니다.

테스트 준비

기존 코드를 철저히 테스트하
고 100% 커버리지를 목표로
합니다.

단계적 업그레이드

한 번에 한 버전씩 순차적으로
업그레이드하여 위험을 줄입
니다.

호환성 해결

deprecated 경고를 해결하고
새로운 API로 코드를 리팩토링
합니다.

마무리 및 Q&A

핵심 정리

Django는 안전하고 확장 가능한 웹 애플리케이션을 빠르게 개발할 수 있는 완전한 프레임워크입니다.

학습 리소스

공식 문서, Django Girls 튜토리얼, Two Scoops of Django 책을 참고하세요.

커뮤니티

Django Forum, StackOverflow, 지역 밋업에 참여하여 질문하고 지식을 나눠보세요.

